

SLSC: Sparse Logic State Compression for Multi-Turn Mathematical Reasoning

Anonymous ACL submission

Abstract

This document is a supplement to the general instructions for *ACL authors. It contains instructions for using the L^AT_EX style files for ACL conferences. The document itself conforms to its own specifications, and is therefore an example of what your manuscript should look like. These instructions should be used both for papers submitted for review and for final versions of accepted papers.

1 Introduction

Multi-turn interaction is rapidly becoming the dominant mode in which language models are deployed, yet recent work has documented substantial performance degradation in this regime: (?) report an average accuracy drop of 39% across six generation task—including mathematics—over 200,000 simulated conversations with 15 top open- and closed-weight LLMs, characterizing this as a “lost-in-conversation” failure. We study a specific and under-examined instance of this broader phenomenon: *multi-turn mathematical reasoning with explicit numerical dependencies*, in which each sub-question is fully specified, but its solution depends on the numerical result of the preceding turn. This setting captures a wide range of practical quantitative tasks—financial calculations, multi-step engineering analyses, scientific computations—that decompose naturally into chained sub-problems. It is particularly acute for small language models (SLMs, $\sim 1\text{B}$ – 10B parameters), which are increasingly deployed under cost and latency constraints but whose limited attention capacity leaves them vulnerable to errors that compound across turns.

The central technical question in this regime is how mathematical intermediate state should be maintained across turns: which equations, constraints, and partial results from prior reasoning must remain accessible to subsequent turns, and in what representational form. Current ap-

proaches uniformly store this state as unstructured text. Prompt compression methods such as LLM-Lingua (?) operate at the token level, agnostic to which tokens encode mathematical constraints. Reasoning-summarization methods such as Reflexion (?) and InfTyThink (?) condense prior reasoning into natural-language narratives in which numerical constraints stated in passing receive no privileged representation. Raw context accumulation simply defers the problem until attention can no longer locate the relevant constraint—a fragility consistent with ?, who show that language models do not robustly use information in long input contexts, with utilization varying sharply by position. We argue these approaches share an assumption—that reasoning state should be flat text—that is poorly matched to mathematical reasoning, where the state needed across turns is intrinsically typed: invariants, active variables, binding equations and inequalities, invalidated paths, and the remaining reasoning gap, each with a distinct role in subsequent inference.

We propose *Sparse Logic State Compression* (SLSC), a twin-agent framework in which a reasoning agent $\mathcal{A}$ generates natural-language chains of thought while a curator agent $\mathcal{B}$ periodically distills accumulated context into a structured state object consisting of atomic mathematical propositions. The reasoning agent’s working context is then replaced by this object, giving each constraint a persistent and addressable location rather than leaving it to be recovered through attention over an ever-growing context. Figure 1 contrasts the two representations.

To evaluate SLSC, we construct a 500-problem multi-turn mathematical reasoning dataset with explicit numerical-dependency annotation by decomposing problems from MATH-500 (?) into chains of dependent sub-questions. We compare three context-maintenance strategies—No Compression (raw accumulation), Natural-Language Summary(a

Baseline: Natural-Language Summary	SLSC: Sparse Logic State
<i>Flat text; intermediate constraints embedded in prose.</i> Context passed to Agent $\mathcal{A}$ at Turn 3: “To approach this problem, I first considered the divisors of $n + 7$ and $2n + 1$. By examining the linear combination $2(n + 7) - (2n + 1)$, I observed that this simplifies to 13, which means any common divisor d must divide 13. Since 13 is prime, the possible values are $d = 1$ or $d = 13$. I then proceeded to analyze the case $d = 13$, which requires $n + 7 \equiv 0 \pmod{13}$, leading to $n \equiv 6 \pmod{13}$. Combined with the bounds, I narrowed down to candidates $n = 6, 19, 32, \dots$, verifying each against the secondary constraint $\dots$” ~ 185 tokens • no addressable structure • key fact mid-sentence	<i>Atomic propositions; each independently addressable.</i> Context passed to Agent $\mathcal{A}$ at Turn 3: 1. d divides $n + 7$ 2. d divides $2n + 1$ 3. $2(n + 7) - (2n + 1) = 13$ 4. d divides 13 5. $d \in \{1, 13\}$ 6. If $d = 13$: $n \equiv 6 \pmod{13}$ 7. Candidates: $n \in \{6, 19, 32, \dots\}$ ~ 55 tokens • each fact retrievable • persistent slot per constraint

Figure 1: Comparison of context representations at Turn 3 of a multi-turn mathematical reasoning task. **Left:** Natural-language summary embeds constraints in prose, requiring attention over unstructured text to locate key facts (highlighted). **Right:** SLSC maintains atomic propositions in numbered slots, giving each constraint a persistent and addressable location. SLSC reduces token count by $\sim 70\%$ while improving constraint retrievability.

Reflexion-style condensed narrative), and SLSC—paired with reasoner models spanning four size from the Qwen3.5 family () (0.8B, 2B, 4B, 9B), with a fixed gemma-3-27b-it () curator. Across this grid, SLSC improves final-answer accuracy by [Z%] over No Compression and [Z’%] over the natural-language summary baseline while reducing per-turn token consumption by [W%]. Critically, the accuracy gap between SLSC and No Compression [widens / narrows] as reasoner size [decrease / increase], indicating that the limited attention capacity of SLMs make them disproportionately reliant on structured state maintenance for chained mathematical reasoning.

We make four contributions:

1. We construct, to our knowledge, the first multi-turn mathematical reasoning dataset with explicit numerical-dependency structure, complementing existing multi-turn benchmarks that focus on underspecification rather than numerical chaining.
2. We document the severity of context-accumulation failure in multi-turn mathematical reasoning at SLM scale, and show that the benefit of structured state maintenance scales inversely with reasoner size—situating this as a distinctly SLM-scale bottleneck and extending the lost-in-conversation and long-context utilization literature to numerically chained settings.
3. We propose SLSC, a twin-agent state-

compression framework, and show that it improves multi-turn final-answer accuracy by [Z%] and reduce per-turn token consumption by [W%] over raw- context accumulation, with consistent gains across all four evaluated SLM reasoner sizes.

4. Contrary to our initial hypothesis, we find through ablation that the choice of state representation syntax—Lean-style formal language versus a simplified structured schema—does not significantly affect performance: it is the *granularity* of slotted state, not the *formality* of its syntax, that drives the gains. This has practical implications at SLM scale, where simplified schema are substantially more reliable to generate than formal-language ones.

2 Related work

Multi-turn reasoning degradation. Recent work documents systematic performance loss in multi-turn settings. ? report an average 39% accuracy drop across six generation tasks over 200,000 simulated conversations with 15 frontier models, characterizing failure as a “lost-in-conversation” phenomenon. ? show that language models do not robustly use information in long contexts, with retrieval accuracy varying sharply by position. These findings establish multi-turn degradation as broad-spectrum, but existing multi-turn benchmarks largely target underspecification and clarification rather than numerical chaining. Our

dataset complements this literature by isolating numerical-dependency failures, where each sub-question is fully specified but its solution requires the prior turn’s numerical result.

Prompt and token-level compression Prompt compression methods reduce input length while attempting to preserve task performance. LLM-Lingua (?) and its extensions identify and remove low-information tokens via small auxiliary models, operating at the lexical level without awareness of which tokens encode task-critical structure. By design, a numerical constant inside an inequality and a discourse adjective receive comparable treatment. SLSC differs along two axes: compression operates over semantic units—atomic mathematical propositions—rather than tokens, and is performed by a dedicated curator agent prompted to identify mathematical content. We view the two approaches as orthogonal and do not include token-level compression in our experimental comparison, which focuses on state-maintenance strategies sharing the same multi-turn decomposed setup.

Reasoning-state compression and summarization A closer line of work compresses intermediate reasoning state rather than raw inputs. Reflexion (?) condenses prior trajectories into natural-language reflections that condition subsequent attempts. InfyThink (?) extends this idea to long-horizon reasoning by iteratively summarizing chain-of-thought and re-prompt the model on the summary, achieving large context savings. Conceptually these methods sit closest to SLSC, and two differences are central. First, both produce flat natural-language summaries in which numerical constraints stated in passing receive no privileged representation; SLSC enforces an atomic-proposition schema in which each targets monolithic single-question reasoning with internally-induced summarization points and assumes a trained summarization checkpoint, whereas our setting is externally-decomposed multi-turn dialogue with explicit numerical dependencies and is fully training-free. A head-to-head empirical comparison would require either retraining InfyThink on our dependency-annotated dataset or rebuilding it as a prompted pipeline; we leave this to future work and restrict our experiments to training-free, setup-compatible baselines.

Formal language augmentation for reasoning. A separate line of work explores formal-language

augmentation for mathematical reasoning. Auto-formalization (?) translates natural-language statements into formal expressions; Draft, Sketch, and Prove (?) interleaves natural-language drafting with formal verification; LeanDojo (?) provides infrastructure for retrieval-augmented theorem proving. These works target end-to-end proof construction or single-statement formalization, with verifier feedback in the loop. SLSC’s Lean 4 augmentation (Section 3.3) instead explores a lighter use of formal language—type-annotating extracted propositions without a verifier—and our ablation (Section 5.2) finds no consistent accuracy gain. This negative result aligns with the broader emphasis on verifier-grounded formalization: absent a Lean kernel check, the type annotation provides no signal that the reasoner can reliably exploit. SLSC thus occupies a specific intersection in this landscape—training-free, multi-turn, mathematics-targeted, and operating on semantic units of intermediate state—with raw context accumulation as the strongest setup-compatible baseline.

3 Method

3.1 Overview

SLSC compresses the accumulated multi-turn reasoning state into a sequence of atomic mathematical propositions, maintained across turns by a curator agent. Given a mathematical problem P decomposed into sub-questions $q_1, \dots, q_T$, we denote the compressed state at turn t as S_t . Our framework operates in two phases per turn t : (1) **Reasoning**: Agent $\mathcal{A}$ generates response $r_t = \mathcal{A}(P, S_{t-1}, q_t)$, conditioned on the compressed state from the previous turn; (2) **Curation**: Agent $\mathcal{B}$ updates the state $S_t = \mathcal{B}(S_{t-1}, r_t)$. Figure ?? illustrates the overall architecture.

3.2 Agent A: The Reasoner

Agent $\mathcal{A}$ serves as the per-turn reasoning component. At each turn t , it receives a structured prompt comprising the original problem P , the current compressed state S_{t-1} , and the sub-question q_t , and produces a natural language response r_t :

$$r_t = \mathcal{A}(P, S_{t-1}, q_t)$$

Agent $\mathcal{A}$ is the frequent-call component of the pipeline, invoked once per turn. Specific model instantiations are described in Section 4.

Algorithm 1 Agent $\mathcal{B}$ Curation: extract atomic propositions from response, optionally formalize each, then merge into compressed state.

Require: S_{t-1}, r_t (current state, Agent $\mathcal{A}$ response)

Ensure: S_t

```

1:  $\mathcal{P}_t \leftarrow \text{EXTRACT}(r_t)$   $\triangleright$  LLM-based proposition extraction
2: for each  $p \in \mathcal{P}_t$  do (optional)
3:    $p.\text{lean} \leftarrow \text{FORMALIZE}(p)$   $\triangleright$  Autoformalize to Lean 4
4: end for
5:  $S_t \leftarrow \text{MERGE}(S_{t-1}, \mathcal{P}_t)$   $\triangleright$  LLM-based deduplication and update
6: return  $S_t$ 

```

3.3 Agent B: The Curator

Agent $\mathcal{B}$ is the core technical contribution of SLSC. It maintains a persistent compressed state by processing $\mathcal{A}$'s response through three sequential sub-steps: extraction, optional formalization, and merging. Algorithm 1 summarizes the pipeline.

3.3.1 Extraction

Given $\mathcal{A}$'s response r_t , the extraction step produces a set of atomic propositions $\mathcal{P}_t = \{p_1, \dots, p_k\}$, where each p_i is a self-contained mathematical claim. We use an LLM-based extractor prompted to identify equations, inequalities, numeric results, and named mathematical properties, while ignoring procedural descriptions and section headers. This yields propositions that are reusable across turns without requiring the full reasoning trace.

The following box shows a representative extraction example on a divisibility problem:

Extraction Example

Input (excerpt from r_t): "Since $d \mid n+7$ and $d \mid 2n+1$, we can compute $d \mid 2(n+7) - (2n+1) = 13$. Therefore d must divide 13..."

Extracted propositions $\mathcal{P}_t$:

1. d divides $n+7$
2. d divides $2n+1$
3. $2(n+7) - (2n+1) = 13$
4. d divides 13

3.3.2 Merging

The merging step integrates the newly extracted propositions $\mathcal{P}_t$ into the existing state S_{t-1} to produce S_t . We apply three LLM-prompted merge rules:

- **Deduplication:** semantically equivalent

propositions are consolidated into a single entry.

- **Supersession:** a new proposition that contradicts or subsumes an existing entry replaces it.

- **Preservation:** propositions with no conflict are retained verbatim.

These rules prevent unbounded state growth while ensuring no established fact is silently overwritten. The merger is implemented as a single prompted call to $\mathcal{B}$; full prompt details appear in Appendix ??.

3.3.3 Formalization

As an optional augmentation, each extracted proposition p may be autoformalized into a Lean 4 statement appended alongside its natural language form. We explore Lean 4 augmentation to assess whether structured logical grounding aids the reasoner. Concretely, $\mathcal{B}$ is prompted to produce a theorem stub for each proposition, with the proof obligation left as sorry. We treat this component as optional: our ablation study (Section ??) shows no consistent accuracy gain from formalization, and all primary results are reported without it unless stated otherwise.

3.4 State Representation

The state S_t is rendered as a numbered list of natural language propositions, optionally paired with Lean 4 stubs, and passed directly to $\mathcal{A}$ as part of its prompt context. Each proposition is atomic and ordered by extraction sequence; inter-proposition dependencies are not encoded explicitly. The state has no hard size cap; deduplication during merging controls growth in practice. The following example shows a typical S_t after three curation steps:

[Known facts from prior steps]

1. d divides $n+7$
theorem prop1 : $d \mid (n+7)$:= by sorry
2. d divides $2n+1$
theorem prop2 : $d \mid (2*n+1)$:= by sorry
3. $2(n+7) - (2n+1) = 13$
theorem prop3 : $2*(n+7) - (2*n+1) = 13$:= by sorry
4. d divides 13
theorem prop4 : $d \mid 13$:= by sorry

3.5 Inference Workflow

Algorithm 2 summarizes the end-to-end multi-turn inference procedure.

Algorithm 2 SLSC Multi-Turn Inference: iterate over sub-questions, alternating Agent $\mathcal{A}$ reasoning and Agent $\mathcal{B}$ state curation, then extract the final answer.

Require: Problem P , sub-questions $\{q_1, \dots, q_T\}$, agents $\mathcal{A}, \mathcal{B}$

Ensure: Final answer

```

1:  $S_0 \leftarrow \emptyset$   $\triangleright$  empty initial state
2: for  $t = 1$  to  $T$  do
3:    $r_t \leftarrow \mathcal{A}(P, S_{t-1}, q_t)$   $\triangleright$  Agent  $\mathcal{A}$  reasons
4:    $S_t \leftarrow \mathcal{B}(S_{t-1}, r_t)$   $\triangleright$  Agent  $\mathcal{B}$  updates state
5: end for
6: return EXTRACTANSWER( $r_T$ )

```

Table 1: Turn-count distribution of the evaluation set ($n = 100$).

Turn count	# Problems	%
2 turns	13	13.0
3 turns	58	58.0
4 turns	26	26.0
5 turns	3	3.0
Total	100	100.0

4 Experimental Setup

4.1 Dataset

We evaluate SLSC on a multi-turn extension of MATH-500, restricted to Levels 3–5. Original single-turn problems are decomposed into sequences of dependent sub-questions through an automated pipeline (described in Appendix ??); each sub-question requires the numerical result of the preceding turn, producing chains of length 2 to 5. Our evaluation set consists of 100 problems with the turn-count distribution shown in Table 1.

4.2 Models

We instantiate Agent $\mathcal{A}$ with four sizes from the Qwen3.5 series (0.8B, 2B, 4B, 9B) to study how reasoner capacity interacts with structured state compression. Agent $\mathcal{B}$ is instantiated with Gemma3-27B throughout. The asymmetric instantiation reflects the differing computational profiles of the two roles: $\mathcal{A}$ is invoked once per turn on the hot path, while $\mathcal{B}$ runs once per turn with a higher per-call latency budget.

All models are served via vLLM. For both agents, we use temperature = 0, top- $p = 1.0$, and maximum generation length of 1024 tokens. We set enable_thinking: True for all Qwen3.5 calls to

keep the configuration consistent across sizes. Detailed sampling parameters and prompt templates appear in Appendix ??.

4.3 Baselines and Conditions

We compare three conditions sharing identical decoding configuration and evaluation pipeline:

- **Full History:** the standard multi-turn baseline. Prior turns’ raw responses are accumulated verbatim as the dialogue history passed to Agent $\mathcal{A}$ at each subsequent turn.
- **SLSC (No Formal):** our method without Lean 4 augmentation. Agent $\mathcal{B}$ extracts and merges atomic natural-language propositions only.
- **SLSC (with Lean):** full SLSC including optional autoformalization of each extracted proposition.

We discuss the relationship to iterative summarization frameworks such as InfyThink (?) in Section ??; direct empirical comparison is precluded by the differing task setups (monolithic single-question reasoning vs. externally-decomposed multi-turn dialogue) and InfyThink’s training requirement.

4.4 Evaluation Metrics

We report four metrics:

- **Fully Correct Rate (FCR)** (primary): fraction of problems for which all sub-question answers are correct.
- **Correct Turn Rate (CTR)** (secondary): fraction of sub-question turns (across all problems) answered correctly.
- **Per-turn Input Tokens:** average input token count seen by Agent $\mathcal{A}$ per turn.
- **Total Inference Tokens:** end-to-end token consumption per problem, including both $\mathcal{A}$ and $\mathcal{B}$ calls.

Answer correctness is determined by exact symbolic match after sympy-based normalization.

4.5 Implementation Details

Experiments are run on [GPU MODEL, E.G., 2× A100 80GB]. Total inference time across all conditions and reasoner sizes is approximately [XX HOURS]. Code, prompts, and dataset construction scripts will be released upon publication.

5 Results and Analysis

5.1 Main Results: SLSC Across Scales

Table 2 reports our main results across four reasoner scales. We observe a clear scaling pattern: at the 4B and 9B scales, SLSC substantially outperforms Full History (+14 and +20 percentage points, respectively); at the 2B scale, SLSC underperforms Full History by 6 points; at the 0.8B scale SLSC provides moderate improvement (+12 points) but absolute accuracy remains low for all conditions. Figure ?? visualizes this non-monotonic pattern.

Table 3 reports Correct Turn Rate, a secondary metric that disentangles per-turn accuracy from cascading errors. The pattern is consistent with FCR: SLSC matches or exceeds Full History at all sizes except 2B (where the gap shrinks substantially compared to FCR, suggesting the 2B FCR deficit is driven partly by failure cascades).

Capability phase transition. The non-monotonicity in Δ_{SLSC} is striking: SLSC fails to benefit reasoners below 4B parameters but provides large gains at 4B and above, with the gain increasing with size. We interpret this as evidence of a *capability threshold* for utilizing structured atomic representations. Smaller reasoners may lack the capacity to parse atomic propositions and reconstruct relevant reasoning context, whereas larger reasoners benefit from reduced context length and the absence of response-style cascading observed in full-history baselines. We provide mechanism evidence in Section 5.4.

5.2 Lean Ablation: A Negative Finding

The Δ_{Lean} column of Table 2 shows that adding Lean 4 formalization to SLSC produces no consistent accuracy improvement: the effect size is at most 2.2 points in absolute value across all four scales, and the sign varies. We investigate why this hypothesized augmentation fails to help downstream reasoning.

We manually annotated 50 randomly sampled Lean outputs produced by Agent $\mathcal{B}$ during the Qwen3.5-4B run (Table 4). Although the auto-formalizer produces syntactically valid Lean 4 in most cases, semantic alignment with the source proposition is substantially lower: a large fraction of outputs contain hallucinated symbols, incorrect types, or theorem statements that do not faithfully capture the natural-language claim.

We identify four contributing factors to the Lean

ablation result:

1. **Out-of-distribution autoformalization:** statement-level autoformalizers are trained on problem-theorem pairs, not on context-stripped atomic intermediate propositions.
2. **No verifier feedback:** SLSC’s autoformalization is stateless; there is no Lean kernel check that could reject malformed output.
3. **Reasoner under-utilization:** qualitative inspection of Agent $\mathcal{A}$ responses shows the reasoner rarely references the Lean stubs explicitly when constructing its answer.
4. **Granularity mismatch:** atomic propositions stripped of problem context lose the surrounding mathematical structure that anchors a meaningful Lean type.

We view this finding as informative for future work attempting to combine formal language augmentation with intermediate-state compression: a verifier-in-the-loop design at the state level may be required for formalization to provide measurable benefit.

5.3 Token Efficiency

A central motivation for SLSC is bounded inference cost across turns. Table 5 compares per-problem token usage at the 4B scale. SLSC substantially reduces Agent $\mathcal{A}$ ’s input context relative to Full History; the reduction is partially offset by Agent $\mathcal{B}$ ’s additional calls, but the end-to-end token cost remains [LOWER / COMPARABLE].

Figure ?? visualizes the per-turn state size growth. Full History grows linearly with turn index, while SLSC maintains a bounded state through deduplication during merging. This bounded-growth property is independent of the accuracy results: regardless of whether SLSC helps or hurts at a given scale, it provides predictable inference cost.

Asymmetric cost distribution. SLSC shifts cost from many small Agent $\mathcal{A}$ calls (on shorter contexts) to fewer but more expensive Agent $\mathcal{B}$ calls. Whether this redistribution is net beneficial depends on deployment constraints—e.g., latency-sensitive applications with tight Agent $\mathcal{A}$ context budgets benefit even when total FLOPs are comparable.

5.4 Failure Mode Analysis

To understand the mechanisms behind both the positive scaling result and the 2B anomaly, we man-

Table 2: Main results across reasoner scales on $n = 100$ problems. FCR = Fully Correct Rate. Δ_{SLSC} = SLSC No Formal – Full History. Δ_{Lean} = SLSC with Lean – SLSC No Formal. Bold indicates the largest deltas per direction. † marks deltas with $p < 0.05$ (McNemar’s test).

Reasoner ($\mathcal{A}$)	Full History	SLSC No Formal	SLSC + Lean	Δ_{SLSC}	Δ_{Lean}
Qwen3.5-0.8B	10.1	22.2	20.0	+12.1 †	−2.2
Qwen3.5-2B	51.0	45.0	44.0	−6.0	−1.0
Qwen3.5-4B	57.0	71.0	72.0	+14.0 †	+1.0
Qwen3.5-9B	60.0	80.0	80.0	+ 20.0 †	0.0

Table 3: Correct Turn Rate (%) across reasoner scales.

Reasoner	Full Hist.	SLSC NF	SLSC + Lean
Qwen3.5-0.8B	21.5	44.3	43.9
Qwen3.5-2B	68.3	67.4	66.1
Qwen3.5-4B	77.7	86.5	86.2
Qwen3.5-9B	81.8	90.3	90.9

Table 4: Manual quality analysis of 50 randomly sampled Lean 4 outputs from SLSC + Lean (Qwen3.5-4B).

Quality dimension	Count	%
Syntactically valid Lean 4	XX	XX.X
Semantically aligned with NL	XX	XX.X
Hallucinated symbols/types	XX	XX.X
Successfully type-checks	XX	XX.X

ually analyzed failed problems on Qwen3.5-4B (where Full History fails) and Qwen3.5-2B (where SLSC fails).

5.4.1 Why does Full History fail at 4B?

We annotated 30 randomly sampled problems on which Full History fails but SLSC succeeds, at the 4B scale. Table 6 reports the failure mode distribution.

The dominant failure mode is **response-style cascade**: Agent $\mathcal{A}$ ’s prior-turn responses, when accumulated in dialogue history, induce the model to produce increasingly verbose meta-reasoning trace in subsequent turns. This consumes the generation budget and, in many cases, prevents the final answer from being produced before `max_tokens` is reached.

Case study. Figure 2 shows a representative example. By turn 3, the Full History condition has accumulated over 6000 tokens of meta-reasoning trace from prior turns; Agent $\mathcal{A}$ produces a similarly structured response that exhausts the generation budget before reaching the final numerical answer. SLSC’s compressed state contains only the relevant atomic facts, allowing the reasoner to produce a direct answer.

Table 5: Token usage per problem at the Qwen3.5-4B scale. $\mathcal{A}$ Input averages across all turns. $\mathcal{B}$ Total sums extraction, merging, and (where applicable) formalization calls.

Condition	$\mathcal{A}$ In	$\mathcal{A}$ Out	$\mathcal{B}$ Total	Combined	Saving
Full History	3058	2110	0	XXXX	—
SLSC No Formal	1153	XXX	XXXX	XXXX	XX%
SLSC + Lean	1153	XXX	XXXX	XXXX	XX%

Table 6: Failure mode distribution of Full History on Qwen3.5-4B (30 problems where SLSC succeeds, Full History fails). Categories are not mutually exclusive; rows may sum to $>100\%$.

Failure mode	Count	%
Response-style cascade	XX	XX.X
Answer truncation at <code>max_tokens</code>	XX	XX.X
Numerical condition forgetting	XX	XX.X
Logic error unrelated to context	XX	XX.X
Other	XX	XX.X

5.4.2 Why does SLSC fail at 2B?

We similarly annotated problems where SLSC fails but Full History succeeds, at the 2B scale (Table 7). The dominant pattern is qualitatively different: 2B reasoners struggle to parse the structured atomic format productively. In several cases the reasoner ignores the state entirely and re-derives answers from the original problem statement only—losing the benefit of accumulated intermediate facts.

This pattern supports our capability-threshold interpretation of Section 5.1: structured atomic representations require a minimum parsing capability from the reasoner that 2B models do not reliably exhibit, even though they handle short raw conversation histories adequately.

5.5 Per-turn Accuracy Breakdown

Figure ?? shows per-turn Correct Turn Rate at the 4B scale. The gap between SLSC and Full History widens at later turns: turn 1 and 2 accuracy is comparable across methods, but Full History accuracy drops sharply at turns 3 and 4 while SLSC

[Case study figure: 4B Full History cascade]
Insert truncated trace comparison here.

Figure 2: Representative 4B failure case. Full History (left) accumulates meta-reasoning trace across turns and truncates before producing the final answer; SLSC (right) maintains a compact atomic state and answers directly.

Table 7: Failure mode distribution of SLSC on Qwen3.5-2B (problems where Full History succeeds, SLSC fails).

Failure mode	Count	%
Fails to parse atomic propositions	XX	XX.X
Ignores state, re-derives from P	XX	XX.X
Misinterprets state semantics	XX	XX.X
Other	XX	XX.X

remains stable. This is consistent with cumulative cascade effects in Full History; SLSC’s bounded state prevents this degradation.

6 Conclusion

This is conclusion part.

Limitations

This document does not cover the content requirements for ACL or any other specific venue. Check the author instructions for information on maximum page lengths, the required “Limitations” section, and so on.

References

A Example Appendix

This is an appendix.